ELECY591

Report : In-painting project

Authors :

BAECKE Erik

PIETRASZ Kamil

USOVITCH Daniel

Teacher :

DELIGIANNIS Nicolaos

Teacher Assistants :

RODRIGO Esther

DI BELLA Leandro

8/06/2025

Contents

Introduction **1**

1 State of the art **2**

2 Description of the code **4**

3 Results And Comparisons **6**

4 Conclusion **10**

5 Sources **11**

Introduction

Due to the recent rise of machine learning models, several useful applications were discovered and implemented. One of them is image in-painting. This technique is useful to reconstruct images where there is some missing data.

This short report will begin by a quick overview of state of the art image inpainting techniques. In the next section, a brief explanation of the code and the different architectures is done. And the last section before the conclusion shows the results obtained with the code.

1

State of the art

Current methodologies for image completion (inpainting) are based on a series of deep neural network architectures, each responding to specific challenges in the reconstruction task.

In image inpainting, we have what is called a mask. The mask represents the missing data or pixels of the image and the goal of in-painting is to recover those missing pixels by using a well-known machine learning technique : Convoluted Neural Networks (CNN's). The CNN's, which applies convolution kernel to extract features hierarchies directly from pixel data. In our implementation, for example, we use 3x3 filters with a stride of 1 and a padding of 1.

A first complete architecture for this task is the auto-encoder. This is an unsupervised learning model that learns efficient data representations. It consists of an encoder that maps the input to a low-dimensional latent space (the bottleneck), and a decoder that reconstructs the input from this latent space. In this project, our auto-encoder is implemented by simplifying the U-Net architecture, specifically by omitting the concatenation step.

The U-Net architecture represents a major development, initially proposed for biomedical image segmentation. It consists of a contraction path (encoder) and an expansion path (decoder). The contraction path applies blocks of successive convolution followed by pooling operations in order to reduce the spatial dimension, while increasing the number of features channels (for example, from 1 or 3 initial channels to 64, 128, then 256 in the bottleneck of our "tiny" model). The key innovation of the U-Net lies in its skip connections. They concatenate the feature maps of the contraction path with the corresponding maps of the expansion path. This mechanism enables high-resolution contextual information, which would otherwise be lost during subsampling, to be transferred directly to the decoder. It is "memory preservation" that enables precise location and fine detail reconstruction.

The default behaviour in these architectures, is to use standard convolution. This treats all pixels in the same way. When a convolution filter passes over a masked area (where pixels are often set to zero), it interprets these zeros as valid information. This processing can "pollute" feature maps by propagating erroneous information and introducing visual artefacts into the reconstructed image, particularly near the edges of the mask. Thus, we need to find an other alternative.

To specifically address the problem of irregular masks, partial convolution, as proposed in the article 'Image Inpainting for Irregular Holes Using Partial Convolutions', was used. The principle is to condition the convolution operation solely on the valid (non-masked) pixels inside the filter window. The output is the re-normalized to take account only of the valid area, and a new mask is generated for the next layer. This approach makes the convolution process agnostic to the shape and location of missing pixel.

Finally, to improve semantic realism and texture quality, generative models such as U-Nets are often trained within a generative adversarial network (GAN). This configuration introduces a discriminator, a second network whose objective is to solve a binary classification task : distinguishing the real image in the dataset from the false images produced by the generator (the Inpainting model). The generator is then trained not only to minimise the reconstruction error, but also to "cheat" the discriminator. It should be noted that in such architectures , regularisation techniques such as batch normalisation are often avoided, as they can introduce artefacts and their effect is negligible with small batch sizes, as is often the case in practice.

2 Description of the code

In this project, we used the DIV2K dataset that is already split in training set and validation set (800 and 100 images respectively). To make the computation faster, we imported the lowest resolution sets (x4 downscaling) and further downscaled them to 128x128 pixels. The option to convert images to greyscale was also added.

Afterwards, masks functions were created : the first one creates a random mask and the second one creates a mask at the centre of the image (1/4 width and height). For training, only the second one was used, but the user is free to use the first function as well for training. The masks are then multiplied with the images to remove some pixels i.e. the mask has the same shape as the image, with zeros for the erased pixels (else ones). Masked images are plotted to verify the masks implementation.

Now, for the main part, different model architectures were implemented : U-Net with partial convolution, U-net without partial convolution and an auto-encoder. For the U-net with partial convolution, two different model sizes were implemented : tiny and medium. In the tiny version we downsample two times (double partial convolution with 3x3 filter, stride = 1 and padding = 1, followed by pooling). Each downsample is preceded by a double convolution layer. This resulting in the number of channels equal to 256 at the bottleneck layer. During the double convolution, the number of channels is only increased in the first convolution. Then, we upsample two times again to close the net following a U shape.

The following "equation" shows how the number of channels increase with the downsampling/upsampling layers. We begin with one channel if we are in greyscale and three in RGB.

$$1(3) \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 1(3) \tag{2.1}$$

On the other hand, notice that during down sampling, we keep the number of channels constant but it is the number of features (here width and height of images) that is reduced by a factor of two (and vice versa for up sampling, the features are increased back again by a factor of two).

In the medium case, instead of downsampling and upsampling two times, we do it four times. The number of channels evolves to 1024 at the bottleneck. This medium model is a lot more computationally heavy which would take a considerably bigger amount of time to train but would lead to better results. The

medium model contrary to tiny models, was not used for training although being implemented in the code.

For the U-Net without partial convolution, the same structure as the tiny U-Net with partial convolution was used. Finally, for the auto-encoder model, we simply skip the concatenation step of the upsampling in U-Net.

Then, we simply create the discriminators (binary true/false images classification) for our GAN before training the model. For the auto-encoder we do a generator only training while for the U-Nets, it is a GAN training (Generator + Discriminator).

For the adversarial loss L_{adv} we used binary cross entropy with logits, as for the reconstruction loss L_{rec} we used the L1 norm loss. A loss dedicated to masks has been additionally used for the generator, which can be described as follows:

$$L_{mask} = \frac{\sum_{ij} \|G - X\|_1 (1 - M)_{ij}}{\sum_{ij} M_{ij}} \tag{2.2}$$

Where $G \equiv G(X \odot M)$ represents the generated output from the generator, which takes as input the image X, considering the applied mask M to it (with an element wise product $\odot$). Thus the loss for the generator is:

$$L_{gen} = (1 - \alpha) L_{adv}(1, D(G)) + \alpha((1 - \beta) L_{rec}(X, G) + \beta L_{mask}) \tag{2.3}$$

Where typically the reconstruction weight $\alpha \approx 0.9$ and the mask weight is usually set around $\beta \approx 0.9$. As for the discriminator's D loss L_{disc}, the adversarial loss L_{adv} is used to calibrate both real with label constant tensor 1, and fake images (the ones generated by the generator) with constant 0 tensor

$$L_{disc} = 0.5 L_{adv}(1, D(X)) + 0.5 L_{adv}(0, D(G)) \tag{2.4}$$

After the training, the parameters are saved and in-painted images can be plotted along with the evolution of the loss in function of the number of epochs.

3 Results And Comparisons

The full results of every tested model:

- UNetP (with partial convolution)
- UNet (original UNet with normal convolution)
- AutoEncoder without discriminator (contrary to previous two GAN models)

They can be found in the folder titled "my_models" containing:

- the models (which can be loaded and infered in inpainting.ipynb): generator (and discriminator)
- the loss functions with respect to epochs for the respective models
- some output results resulting from the inference using the checkpoint with lowest validation loss (not necessarily corresponding to the last epoch due to overfitting): "Output.png"

Curiously, no matter which model was tested, the lowest validation loss for the generator achieved was around 0,75 (before the model starts to overfit). Meanwhile for the discriminator, both the validation and training loss oscillated around 0,7. The output results for the most 'complex' model (i.e. UNet with partial convolutions) are displayed in Figure 3.1, the generator and discriminator loss are respectively in Figure 3.2 and Figure 3.3.

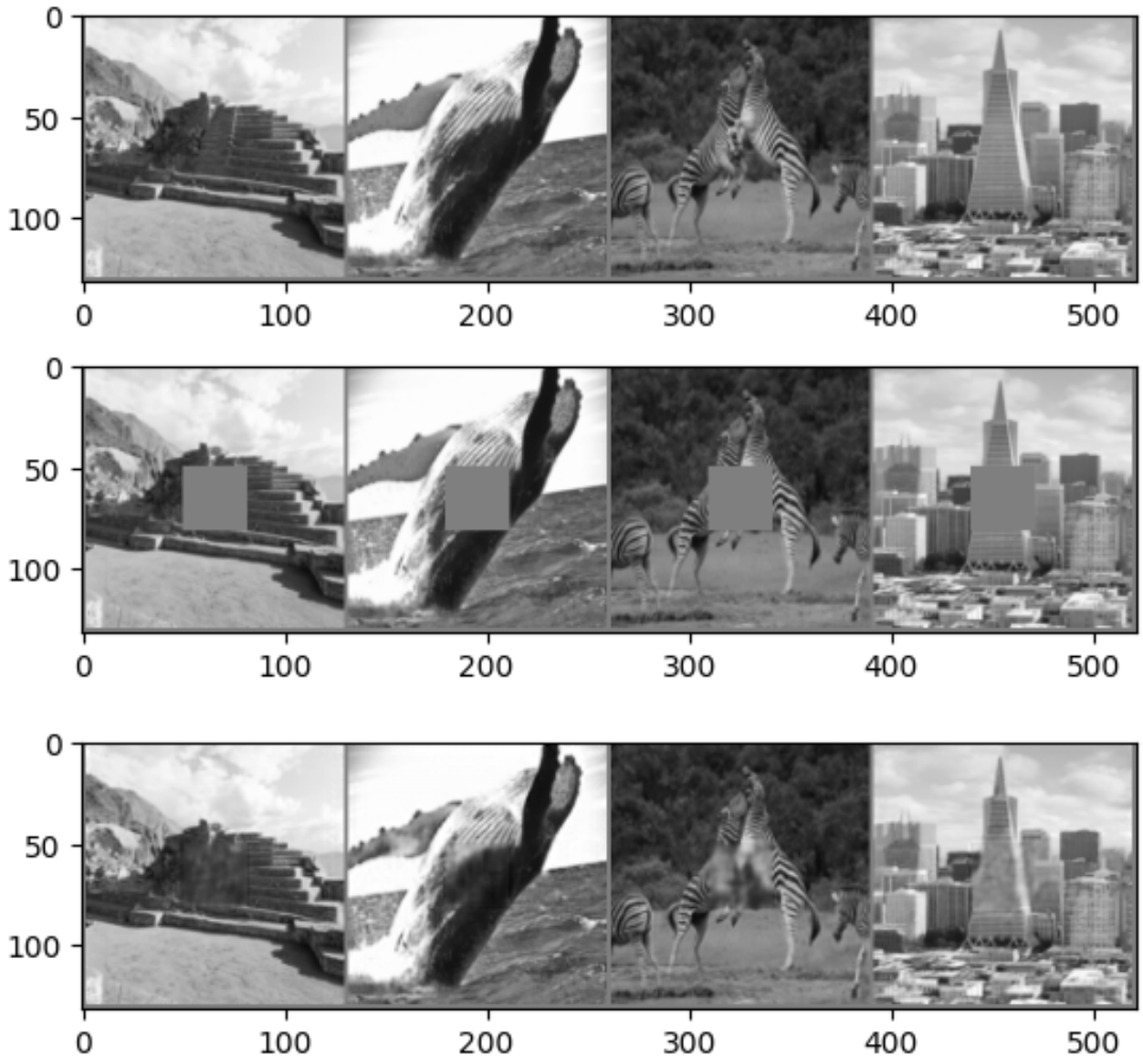


Figure 3.1: Results of inference for the UNet with partial convolution after 50 epochs. First row showcases the original images, second row displays the applied mask which the image as a whole is given as input to the model, the third row is the output of the model. To avoid unnecessary computation time, the images to be trained on were specifically chosen to minimize the number of features i.e. greyscale instead of colours thus requiring 1 channel instead of 3, and low resolution scaled down to 128 x 128 pixels. In practice, higher number of features will not impact anything but will only increase the training time. To obtain better results it is also possible to simply extend the layers, with increasing number of channels (and further feature reduction as we propagate along the encoder type) of the used models, hence even though it was trained on a UNet (using partial convolutions) titled as "tiny" (with 256 out channels in bottleneck) in "inpainting.ipynb", it is also possible to acquire better results with a model, which was not trained, titled as "medium" (this time with 1024 out channels in bottleneck).

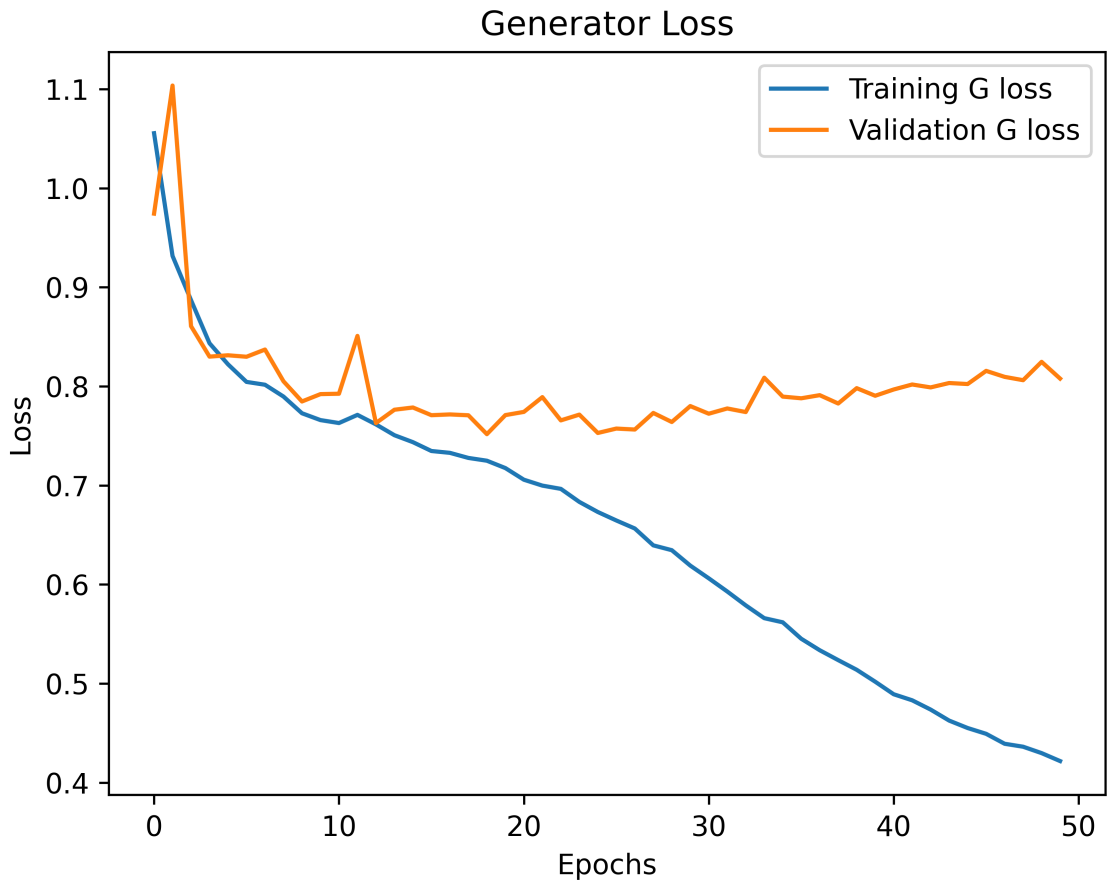


Figure 3.2: Generator loss for the UNet with partial convolution trained for 50 epochs. The lowest achieved loss is around 0,75 no matter which other model was tested (UNet with normal convolution, or AutoEncoder without discriminator)

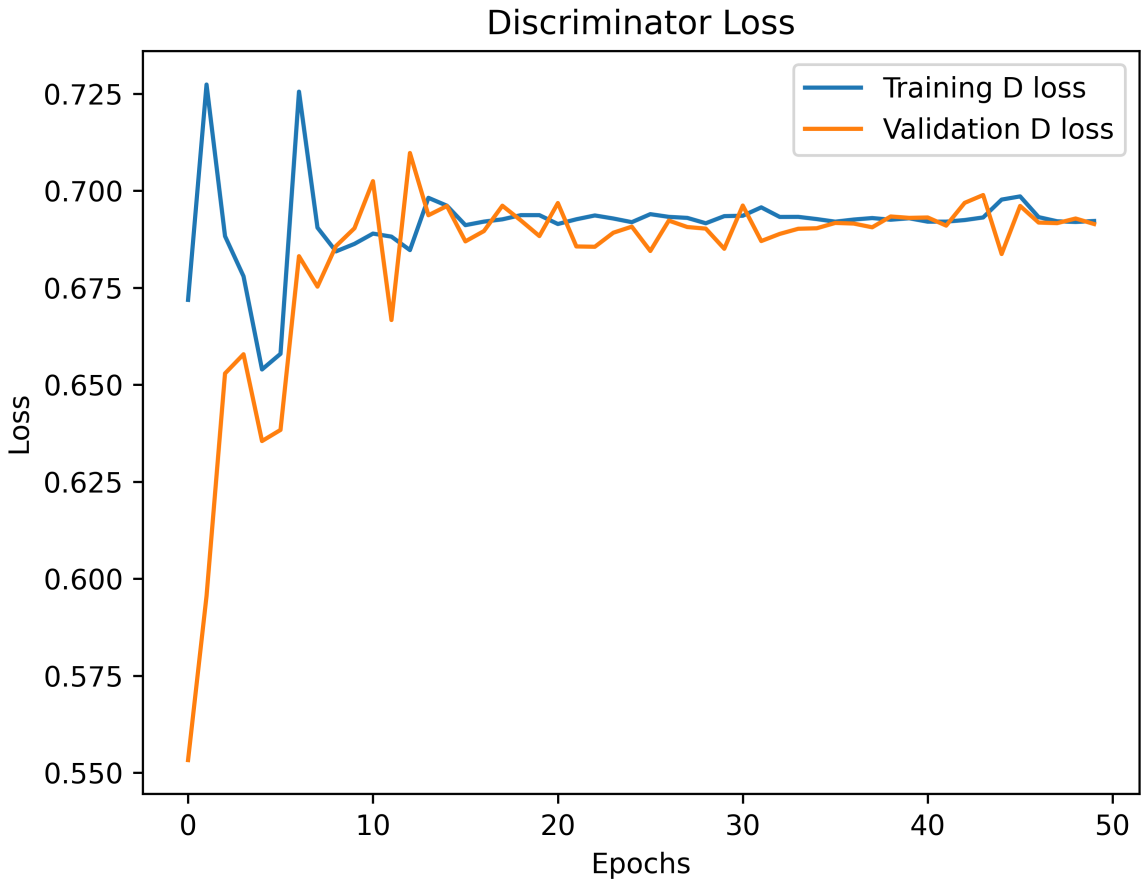


Figure 3.3: Discriminator loss for the UNet with partial convolution trained for 50 epochs. The loss seems to oscillate around 0,7 again displaying similar results no matter which model was used. It could also potentially indicate that the discriminator was not trained for long enough relative to the training steps of the generator which could require for example 5:1 ratio (discriminator vs generator training steps).

4 Conclusion

This project explored the application of different deep learning models for image completion (in-painting). The goal was to reconstruct missing data, represented by a masks, using the convolution neural network (CNNs). In order to achieve this, we implemented and compared with three distinct architectures : an auto-encoder, a U-Net with classical convolutions U-Net with partial convolutions.

Our experiments, carried out on the DIVK2 dataset with images resized to 128x128 pixels and converted to greyscale in order to speed up the calculation and runtime, showed that the different models achieve similar performance in terms of generator validation loss, which stagnates at around a value of 0.75 before overfitting becomes apparent. Similarly, the discriminator loss for the GAN model hovered around 0.7. However, the auto-encoder was trained alone, while the U-Net models benefited from GAN-type training with discriminator.

The U-Net model with partial convolutions, although more complex, did not show any significant improvements under these tests. It is important to note that we mainly used a "tiny" version of this model for reasons of computing time. A heavier version, called " medium ", was considered but not trained; it could potentially offer better results thanks to greater number of channels in the hidden layers .

In conclusion, although the visual results obtained demonstrate a functional capacity for image completion, the loss metric suggest that the performance of the models is comparable in the context of our study. Future work could explore training deeper models such as the "medium" version of U-Net and adjust the training hyper-parameters, notably the ratio of training steps between the generator and the discriminator, to potentially improve the quality of reconstruction.

5 Sources

As a disclaimer, class "PartialConv2d" in "inpainting.ipynb" was taken from Implementing Inpainting GANs which original source actually traces back from NVidia class PartialConv2d(nn.Conv2d). Partial convolutions originated from Image Inpainting for Irregular Holes Using Partial Convolutions: it takes into account missing pixels from applied mask to images, especially relevant in *image inpainting*. UNet was introduced in U-Net: Convolutional Networks for Biomedical Image Segmentation which basically consists of an AutoEncoder with additional concatenation connecting each layer before and after the bottleneck which share the same level (at which the features are encoded/decoded) in a way preserving memory. Another honorable source explaining the state of the art of *image inpainting* is Michał Kudelski (TCL): Inpainting using Deep Learning: from theory to practice, in which it is also explained how applying regularization techniques such as batch normalization or drop outs, are not currently used due to: potentially leading to chromatic aberration (or other artifacts upon image generation); additionally since it is usually trained on low batch of 4, the batch normalization will have a negligible effect.